Compte-rendu

TP N° 20

Prénom : EL Hairche
Nom : Mohamed
Groupe : DEVOAM202

Année scolaire : 2026/2027

Introduction

Ce TP a pour objectif de configurer un projet Android multi-modules avec Gradle. Le projet est divisé en trois modules : app (l'application principale), data (la gestion des données) et domaine (la logique métier). Cette organisation rend le code mieux structuré, plus facile à maintenir et améliorer le temps de compilation. Le TP montre aussi comment définir les dépendances entre les modules et centraliser les configurations communes pour éviter la répétition.

Exercice 1 :

Étapes :

1. Création des modules :

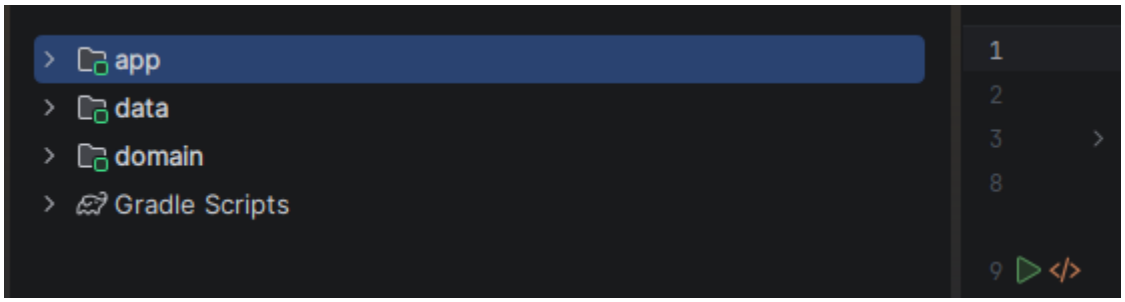


Figure1 : Creation des module

Le projet est créé avec le module par défaut app. Les deux autres modules, data et domain, sont ajoutés avec File > New > New Module dans Android Studio. Chaque module possède son propre fichier build.gradle et ses propres dossiers de code.

2. Configurer les dépendances entre les modules :

```
include( ...projectPaths = ":app", ":data", ":domain")
```

Figure2 : inclure les trois modules

Le fichier settings.gradle déclare les modules qui font partie du projet avec la ligne include ':app', ':data', ':domain'. Sans cette déclaration, Gradle ne reconnaît pas data et domain et ne les compile pas.

3. Configurer le module app :

```
dependencies {
    implementation(project( projectPath = ":data"))
    implementation(project( projectPath = ":domain"))
}
```

Figure3 : Configuration du module app

Dans app/build.gradle, le bloc dependencies contient implementation(":data") et implementation(":domain"). Ces deux lignes permettent au module app d'utiliser les classes de data et de domain. Le mot-clé implementation signifie que ces modules sont utilisés en interne et ne sont pas exposés aux autres modules, ce qui accélère la compilation.

4. Configurer les dépendances des autres modules :

```
dependencies {
    implementation("androidx.room:room-runtime:2.6.1")
}
```

Figure4 : Dépendances propres à chaque module

Chaque module reçoit uniquement les bibliothèques dont il a besoin. Le module data ajoute Room (androidx.room:room-runtime:2.6.1) pour travailler avec la base de données, et le module domain ajoute Kotlin Coroutines (kotlinx-coroutines-core:1.6.4) pour la logique métier asynchrone. Cette séparation garde chaque module léger et indépendant.

5. Créer une configuration commune :

```
android {
    compileSdk = rootProject.extra["compileSdk"] as Int
    defaultConfig {
        minSdk = rootProject.extra["minSdk"] as Int
        targetSdk = rootProject.extra["targetSdk"] as Int
    }
}
```

Figure5 : Configuration commune

Dans le build.gradle à la racine du projet, le bloc extra.apply définit des variables partagées : kotlin_version (1.8.0), compileSdk (34), minSdk (29) et targetSdk (34). Dans chaque module, ces valeurs sont récupérées avec rootProject.extra[“minSdk”] as Int, par exemple. Ainsi, pour changer une version, il suffit de modifier une seule ligne à la racine au lieu de modifier chaque module.

Conclusion

Ce TP m'a permis de comprendre l'intérêt de l'architecture multi-modules dans un projet Android. La séparation en app, data et domain rend le code plus organisé et plus facile à maintenir. Le fichier settings.gradle déclare les modules, les dépendances implementation relient app aux autres modules, et chaque module garde ses propres bibliothèques. Enfin, la configuration commune à la racine centralise les versions du SDK et de Kotlin, ce qui évite la redondance et simplifie les mises à jour.